

Algoritmi-Modulo algoritmi per bioinformatica

Analisi probabilistica e algoritmi randomizzati-Capitolo 5 CLRS

Roberto Posenato

versione 2026-1, 15/03/2026

- 1 Preliminari di calcolo delle probabilità
 - Variabili casuali indicatrici
- 2 Analisi probabilistica degli algoritmi
 - Problema delle assunzioni
- 3 Algoritmi randomizzati
 - Idea generale
 - Permutazione casuale degli array
 - Esercizi svolti
- 4 Esercizi da svolgere

*“Tell me and I forget, teach me and I remember,
involve me and I learn.”*

Benjamin Franklin

*“A student who achieves knowledge through free investigation
and spontaneous effort will be able to retain that knowledge
and will have acquired a methodology that can serve for a li-
fetime.”*

Jean Piaget

Scopo della lezione

- Introdurre all'analisi probabilistica della complessità degli algoritmi come alternativa all'analisi del caso peggiore.
- Introdurre gli algoritmi *randomizzati*.

Nota!

In questa lezione si assume che lo studente conosca il concetto di *variabile casuale* (calcolo delle probabilità).

Per chi non lo conosce, è sufficiente, ai fini di questa lezione, studiare l'Appendice C [CLRS].

Preliminari

Variabili casuali indicatrici

- Una **variabile casuale (v.c.)** discreta è una funzione che associa a un evento un numero reale.
- La **funzione di probabilità** di una v.c. X è definita come $f(x) = \Pr\{X = x\}$.
- Il **valore atteso** (media/aspettazione) è $E[X] = \sum_x x \Pr\{X = x\}$.
- *Linearità del valore atteso*: $E[cX + dY] = cE[X] + dE[Y]$ (vale anche se X e Y sono dipendenti).
- $E[XY] = E[X]E[Y]$ solo quando X e Y sono indipendenti.

Esempio 1

Si consideri il gioco: “lancio di due monete uguali. Si vincono 3 euro per ogni testa e si perdono 2 euro per ogni croce.”

- X è la v.c. “*valore della vincita*”.
- Probabilità che si vincano 6 euro (2 teste): $\Pr\{X = 6\} = 1/4$
- $E[X] = 6 \Pr\{2T\} + 1 \Pr\{1T, 1C\} - 4 \Pr\{2C\} = 6 \cdot \frac{1}{4} + 1 \cdot \frac{1}{2} - 4 \cdot \frac{1}{4} = 1$.

Preliminari

Variabili casuali indicatrici

Le variabili casuali **indicatrici** sono particolari v.c. per le quali il calcolo della media è più semplice.

Definizione 1

Dato uno spazio campionario S e un evento A , la variabile casuale indicatrice $I\{A\}$ associata all'evento A è

$$I\{A\} = \begin{cases} 1 & \text{se } A \text{ si verifica} \\ 0 & \text{altrimenti} \end{cases}$$

Lemma 1

Dato uno spazio campionario S e un evento A , posto $X_A = I\{A\}$, si ha $E[X_A] = \Pr\{A\}$.

Dimostrazione.

$$E[X_A] = E[I\{A\}] = 1 \Pr\{A\} + 0 \Pr\{\bar{A}\} = \Pr\{A\}.$$



- Le v.c. indicatrici rendono più facili le valutazioni di certe medie:

Esempio 2

X = numero di 'Testa' nel lancio di una moneta per n volte.

Stimare $E[X]$.

- È noto che X ha una distribuzione binomiale e che $E[X] = n/2$ (calcolo della sommatoria pesata di n variabili casuali bernoulliane).
- Sia X_i v.c. indicatrice "*Testa all' i -esimo lancio*" $\Rightarrow X = \sum_{i=1}^n X_i$.
- $E[X] = E[\sum_{i=1}^n X_i] = \sum_{i=1}^n E[X_i] = \sum_{i=1}^n 1/2 = n/2$.

Analisi probabilistica degli algoritmi

Problema delle assunzioni

PROBLEMA ASSUNZIONI

INPUT: Un insieme di n candidati. Costo di un colloquio $c_c > 0$, costo di licenziamento-e-assunzione $c_a > 0$ t.c. $c_c \ll c_a$.

OUTPUT: Assumere il miglior candidato possibile con il minor costo possibile. Si può confrontare solo un candidato alla volta con l'impiegato corrente.

Una strategia di ottimizzazione locale (algoritmo di ricerca locale)

- Si assume inizialmente il primo candidato come riferimento corrente; ogni volta che si esegue un colloquio, se il candidato è migliore del corrente, si licenzia il corrente e si assume il nuovo candidato.

Se m è il numero totale di licenziamenti-e-assunzioni fatti per trovare il migliore, il costo totale associato a questa strategia è $n \cdot c_c + m \cdot c_a$ (**stiamo valutando una complessità che non è il tempo!**).

Analisi probabilistica degli algoritmi

Problema delle assunzioni

PROBLEMA ASSUNZIONI

INPUT: Un insieme di n candidati. Costo di un colloquio $c_c > 0$, costo di licenziamento-e-assunzione $c_a > 0$ t.c. $c_c \ll c_a$.

OUTPUT: Assumere il miglior candidato possibile con il minor costo possibile. Si può confrontare solo un candidato alla volta con l'impiegato corrente.

- Qualsiasi strategia (algoritmo) deve costare $n \cdot c_c$. Quindi non lo consideriamo più di tanto.
- Quello che conta è $m \cdot c_a$.
- Nella strategia adottata, m può assumere qualsiasi valore tra 1 (caso migliore) e n (caso peggiore).
- Vogliamo calcolare il costo medio della strategia!
- Da cosa dipende? **Dalla distribuzione dei candidati in input!**

Analisi probabilistica degli algoritmi

Problema delle assunzioni

- Se è nota la distribuzione di probabilità della posizione del candidato migliore all'interno della sequenza, si può fare un'analisi che determina il costo medio: **analisi di complessità nel caso medio**.
- Fare assunzioni sulla distribuzione di probabilità dell'input non è sempre possibile.
- Talvolta si può manipolare l'input per far sì che abbia una distribuzione di probabilità uniforme (**algoritmi randomizzati**).

Analisi probabilistica degli algoritmi

Problema delle assunzioni

Calcolo costo medio problema assunzioni

- Si supponga che non vi siano due candidati con la stessa bravura; quindi i candidati sono totalmente ordinabili rispetto alla bravura.
- Si assuma che ciascuna sequenza di candidati sia equiprobabile. Ogni sequenza ha probabilità $\frac{1}{n!}$
- Sia X = “numero di candidati assunti”. Il costo medio cercato è $c_a E[X]$.
- $E[X] = \sum_x x \Pr\{X = x\}$ non è facile da calcolare.
- Alternativa: si definiscono n variabili casuali indicatrici $X_i = I\{\text{il candidato } i\text{-esimo viene assunto}\}$
- Vale $X = \sum_{i=1}^n X_i$.
- $E[X] = \sum_{i=1}^n E[X_i] = \sum_{i=1}^n \Pr\{X_i = 1\}$.

Analisi probabilistica degli algoritmi

Problema delle assunzioni

Valutazione $\Pr\{X_i = 1\}$

- Il candidato in posizione i viene assunto quando è migliore di tutti i candidati da 1 a $i - 1$.
- I primi i candidati si presentano in ordine casuale, tutti con la stessa probabilità.
- Qualsiasi candidato tra 1 e i ha la stessa probabilità di essere il migliore.
- Il candidato i ha probabilità $1/i$ di essere migliore di tutti i candidati da 1 a $i - 1$.

$$\Pr\{X_i = 1\} = \frac{1}{i}$$

Analisi probabilistica degli algoritmi

Problema delle assunzioni

Conclusione

- $E[X_i] = \Pr\{X_i = 1\} = \frac{1}{i}$
- $E[X] = \sum_{i=1}^n E[X_i] = \sum_{i=1}^n \frac{1}{i} \leq \ln n + 1.$
- Costo medio del problema delle assunzioni
 $\leq c_a(\ln n + 1) = O(c_a \ln n) = O(c_a \lg n)$
- Ricordiamo che il costo nel caso peggiore è $O(c_a n)$.

$\sum_{i=1}^n \frac{1}{i}$ è la serie armonica limitata a n .

Vale l'approssimazione $\ln(n+1) \leq \sum_{i=1}^n \frac{1}{i} \leq \ln n + 1$ [CLRS Appendice A].

Ad esempio, per $n = 100$, $4.615.. \leq 5.177.. \leq 5.605..$

Algoritmi randomizzati

Idea generale

- Come è stato detto, fare assunzioni circa la distribuzione di probabilità dell'input non è sempre possibile o facile (si pensi allo stesso problema delle assunzioni)!
- Per molti problemi, questo impedimento può essere superato facilmente e si può determinare la complessità del caso medio:
“Aniché ipotizzare una distribuzione di probabilità degli input, un algoritmo può **importa riordinando** l'input prima di considerarlo.”
- Un algoritmo che riordina casualmente gli elementi di un input e/o esegue delle scelte in modo casuale è detto **algoritmo randomizzato**.

Esempio 3

Nel problema delle assunzioni, anziché assumere che l'input sia equiprobabile, si modifica l'algoritmo in modo che, a ogni input, esegua una permutazione casuale dello stesso prima d'iniziare la computazione. In questo modo, il valore medio è *aspettato* per *ogni* input e non soltanto per quelli estratti da una particolare distribuzione.

Algoritmi randomizzati

Idea generale

Sempre considerando il problema delle Assunzioni.

Analisi probabilistica

- La strategia (algoritmo) è deterministica. Dando lo stesso input più volte, il risultato **non cambia**.
- L'input con gli elementi ordinati per competenza determina il caso peggiore.
- **Ipotizzando** la distribuzione di probabilità uniforme degli input, il costo (complessità) **medio** è $O(c_a \lg n)$

Strategia (algoritmo) randomizzata

- La strategia adottata permuta casualmente gli elementi dell'input prima di considerarli. Dando lo stesso input più volte, **l'esecuzione può cambiare**.
- Il comportamento dell'algoritmo non dipende solo da un input particolare, ma anche dalle scelte casuali effettuate.
- Non si deve fare nessuna ipotesi sulla distribuzione di probabilità degli input.
- Il costo (complessità) **medio** è $O(c_a \lg n)$.

Algoritmi randomizzati

Permutazione casuale degli array

- Se l'input di un algoritmo è un array, una permutazione casuale opportuna garantisce che la permutazione risultante abbia **distribuzione uniforme**.
- Si assuma che esista un **generatore di numeri casuali** $\text{RANDOM}(a, b)$: ritorna un intero i tale che $a \leq i \leq b$.
- Ciascuno dei valori di $\text{RANDOM}(a, b)$ ha la stessa probabilità di essere generato.
- Tutti i linguaggi di programmazione rendono disponibile una chiamata a un **generatore di numeri pseudo-casuali** che realizza un'implementazione di $\text{RANDOM}(a, b)$.

In Java, un generatore di numeri pseudo-casuali è la classe `java.util.Random`

Algoritmi randomizzati

Permutazione casuale degli array

Una procedura efficiente per costruire permutazioni equiprobabili.

Algoritmo: RANDOMIZEINPLACE(A)

```
1:  $n = |A|$   
2: for  $i = 1$  to  $n$  do  
3:    $\lfloor$  Scambia  $A[i]$  con  $A[\text{RANDOM}(i, n)]$   
4: return  $A$ 
```

- Complessità in tempo $O(n)$.
 - Ci sono n iterazioni e, in ciascuna, si esegue un numero costante di operazioni elementari, incluso uno scambio e una chiamata a $\text{RANDOM}(i, n)$.
 - Si assume che $\text{RANDOM}(i, n)$ abbia costo costante rispetto a n , ipotesi ragionevole nelle implementazioni software.
- Complessità in spazio $O(1)$.

Algoritmi randomizzati

Permutazione casuale degli array

Lemma 2

RANDOMIZEINPLACE genera una permutazione casuale uniforme.

Dimostrazione

Si dimostra un invariante di ciclo e poi la tesi.

Invariante di ciclo:

Prima della i -esima iterazione, per ogni possibile $(i - 1)$ -permutazione, $A[1 : i - 1]$ contiene la permutazione considerata con probabilità $\frac{(n-i+1)!}{n!}$.

- **Base:** appena prima dell'iterazione con $i = 1$, per l'unica 0-permutazione (quella vuota), il sottoarray $A[1 : 0]$ la contiene con probabilità $1 = (n - 1 + 1)!/n!$. La base è quindi verificata.

Algoritmi randomizzati

Permutazione casuale degli array

- **Conservazione:** si assume che prima del ciclo i , ogni possibile $(i - 1)$ -permutazione appaia nel sottoarray $A[1 : i - 1]$ con probabilità $(n - i + 1)!/n!$.
 - Sia $\langle x_1, \dots, x_{i-1}, x_i \rangle$ una permutazione qualsiasi.
 - $\langle x_1, \dots, x_{i-1} \rangle$ compare in $A[1 : i - 1]$ con probabilità $(n - i + 1)!/n!$ per ipotesi induttiva.
 - La probabilità che x_i sia scelto è $1/(n - i + 1)$, perché l'algoritmo lo sceglie uniformemente tra gli $n - i + 1$ elementi rimasti in $A[i : n]$.
 - La probabilità di ottenere quindi $\langle x_1, \dots, x_{i-1}, x_i \rangle$ è

$$\frac{(n - i + 1)!}{n!} \frac{1}{n - i + 1} = \frac{(n - i)!}{n!}$$

- $\frac{(n-i)!}{n!}$ è ancora l'invariante, perché vale prima dell'inizio dell'iterazione successiva $i + 1$.
- **Conclusione:** alla fine del ciclo **for** si ha $i = n + 1$. Applicando l'invariante, ogni possibile n -permutazione compare in $A[1 : n]$ con probabilità

$$\frac{(n - (n + 1) + 1)!}{n!} = \frac{0!}{n!} = \frac{1}{n!}$$

Algoritmi randomizzati

Esercizi svolti

Esercizio 1 (5.2–5 [CLRS])

Usare le variabili casuali indicatrici per risolvere il seguente problema. Ognuno dei n clienti consegna il suo cappello a una persona che si trova all'ingresso del ristorante. La persona restituisce i cappelli ai clienti in ordine casuale.

Qual è il numero atteso di clienti che riceveranno il loro cappello (e non quello di un altro)?

Algoritmi randomizzati

Esercizi svolti

Esercizio 2 (5.2–6 [CLRS])

Sia $A[1 : n]$ un array di interi distinti. Se $i < j$ e $A[i] > A[j]$, allora (i, j) è detta **inversione**. Assumendo che A abbia distribuzione uniforme, usare le variabili casuali indicatrici per calcolare il numero atteso di inversioni.

Algoritmi randomizzati

Esercizi svolti

Esercizio 3 (5.3–4 [CLRS])

Si consideri la seguente procedura

Algoritmo: PERMUTEBYCYCLE(A)

```
1:  $n = |A|$ 
2:  $B[1 : n]$  nuovo array
3:  $offset = \text{RANDOM}(1, n)$ 
4: for  $i = 1$  to  $n$  do
5:    $dest = i + offset$ 
6:   if  $dest > n$  then  $dest = dest - n$ 
7:    $B[dest] = A[i]$ 
8: return  $B$ 
```

Si dimostri che ogni elemento $A[i]$ ha probabilità $1/n$ di trovarsi in una posizione particolare di B . Poi, che la permutazione risultante **non** ha distribuzione uniforme.

Esercizi da svolgere

- Studiare Capitolo 5 fino a 5.4 escluso [CLRS].
- Risolvere gli esercizi 5.2–1, 5.2–3, 5.3–2 (pensare al caso $n = 3$).